

# 低压电气成套智能报价清单生成系统 — 创意提案

## 目录

- 1. 项目摘要
- 2. 问题分析
- 3. 系统架构设计
- 4. 数据模型设计
- 5. 核心算法方案
- 6. 测试方案
- 7. 可视化与交互方案
- 8. 创新点与技术亮点

## 一、项目摘要

### 1.1 背景概述

低压电气成套设备是电力系统的末端核心节点，广泛应用于数据中心、医院、工业厂房、城市综合体、市政设施等几乎所有需要配电的场景。一个典型的成套配电项目——例如一条工厂生产线或一栋医院大楼——往往包含数十台乃至数百台低压开关柜、中压柜、变频柜、MCC控制柜及终端配电箱。每一台柜子内部配置了数十种元器件：断路器、接触器、互感器、继电器、铜排、绝缘件、线缆辅材等等，这些物料的型号规格、品牌、数量、价格直接决定了项目的采购成本和最终报价。

在当前的行业实践中，项目报价人员通常需要同时面对四到五种不同格式的输入资料——电气系统图（DWG或PDF格式）、主元器件清单（Excel格式）、技术配置说明书（Word或PDF格式）、设备清单（可能又是另一份Excel），以及偶尔出现的现场照片或截图。这些资料由不同的工程师使用不同的模板制作，命名习惯各异、格式不统一。报价人员需要手工翻阅每一页图纸、对照每一行清单，凭借多年积累的经验判断柜型对应的典型配置、识别哪些物料存在品牌冲突、哪些属于长交期需要提前采购。这个过程不仅耗时巨大——一个中等规模项目通常需要数天——而且极易出错：漏掉一个物料就意味着采购遗漏，数量算错就会导致成本偏差，品牌写错可能带来现场更换的巨大损失。

### 1.2 系统定位

本系统提供一个可运行的智能解决方案。我们的目标是：让算法完成80%的基础性确定性校验性工作，让工程师聚焦于20%的高价值判断和异常核对。我们计划让系统产出的每一条BOM记录都可以追溯到原始文件的具体位置，工程师可以随时点击查看“这条物料是从哪份清单的第几行抽取的”或“这个柜号是从图纸的哪个区域识别到的”，从而高效完成任务的处理和信息修正。

### 1.3 核心目标

系统围绕以下五个核心目标构建：

- 1.多格式统一解析：对Excel主元器件清单、DWG/PDF电气图纸、Word技术说明书、图片等输入格式建立统一的解析入口，后台分层处理，输出结构化的项目资料索引；
- 2.智能字段抽取：从异构资料中自动识别并抽取柜号、柜型、额定电流、回路数、元器件名称、规格型号、品牌、数量等关键业务字段；
- 3.物料归一化与合并：针对不同项目、不同供应商对同一物料的不同命名方式，通过分层匹配算法实现规范化和同类项合并；
- 4.可追溯BOM与报价生成：按柜体维度生成逐柜BOM明细，跨柜合并生成项目汇总BOM，基于价格优先级策略计算

报价清单，所有结果保留来源证据链； 5.多维度质量校验：自动检测缺项、重复、品牌冲突、长交期风险、柜号不一致、价格缺失等常见质量问题，生成可操作的校验报告；

## 二、问题分析

### 2.1 四个流程

我们从业务角度出发，面向需求分析建立我们的工作流程，以下是模拟真实的业务流程，也决定了我们的方向：

**一、整理资料。** 客户或销售部门发来的询价邮件通常包含若干附件：一份DWG格式的电气系统图、一份Excel主元器件清单、一份Word格式的技术要求说明书，偶尔还有用手机拍摄的现场照片或PDF扫描件。工程师首先需要判断每份文件是什么、是否完整、有无遗漏。

**二、逐柜提取BOM。** 打开Excel清单，按柜号逐行查看：这台进线柜需要哪些元器件？框架断路器是什么品牌什么规格？电流互感器配了几个？辅材（铜排、绝缘子、线缆端子）有没有遗漏？如果清单中某台柜子的信息不全——这是经常发生的情况——工程师需要翻到对应的DWG图纸页面，从系统图中查找补充信息。DWG图纸中，柜体的额定电流可能标注在角落，柜型可能写在图框标题栏中，而元器件的详细规格可能在另一张二次图中。这个过程反复进行，30台柜子可能需要一整天。

**三、物料规范化与汇总。** 不同项目、不同设计师对同一元器件的命名往往不同。需要识别这些是同一种物料，然后在汇总BOM中合并。此外，还需要检查技术说明书中指定的品牌约束——比如“所有断路器必须使用施耐德或ABB品牌”——如果清单中出现了第三个品牌，就需要标记冲突。

**四、报价计算与校验。** 汇总BOM完成后，工程师需要查找每种物料的价格——可能来自历史项目的价格表、供应商报价单或公司内部价格库。对于没有价格的物料，需要标注待询价。最后，逐柜计算小计、汇总项目总价，并进行全面检查：有没有柜号重复？有没有物料数量异常？长交期物料有没有单独标注以便提前采购？

从以上流程可以看出，优化流程的核心问题在于：大量重复性的、规则明确的工作消耗了工程师的绝大部分时间。

### 2.2 五大痛点

我们将以上流程中的困难归纳为六个痛点：

**一：资料类型多** 一个项目可能同时包含DWG图纸、PDF图纸、Excel清单、Word技术说明书。每种格式需要不同的解析能力：DWG需要理解DXF实体模型，PDF需要处理文本层或渲染为图像做OCR，Excel需要处理合并单元格和多级表头，Word需要抽取表格和段落约束。当前没有任何单一工具能统一处理这些格式。

**二：模板不统一** 不同设计院、不同客户、甚至同一公司不同工程师制作的Excel清单模板差异巨大。有的清单把柜号放在第一列、有的放在表头区域、有的甚至用合并单元格跨行表示。列名也五花八门：“设备名称”、“元器件”、“名称及规格”、“Description”指向的可能都是同一个字段。这种非标准化使得任何简单的“按列名读取”策略都无法通用。

**三：关键信息分散** 柜体的完整描述散落在多个文件中：柜号可能在Excel清单中出现，柜型标注在DWG图纸的标题栏中，额定电流写在系统图的进线侧，进出线方式可能在Word技术说明中提及。报价人员需要在多份文件之间来回切换、交叉比对，效率极低且容易遗漏。

**四：变更频繁，影响面难以评估。** 询价阶段客户经常更新技术要求或补充资料。当一份新的Excel清单替换旧版时，工程师需要人工逐行对比找出新增、删除和修改的柜体和物料——三十台柜子、数百行BOM的逐行对比极易产生遗漏。

**五：校验维度多** 报价结果需要检查的维度至少包括：柜号是否缺失或重复？物料数量是否异常？同一柜内是否存在完全重复的物料行？.....人工逐项检查不仅耗时，而且在疲劳状态下难以保证100%覆盖。

## 2.3 六个难题

将上述业务痛点映射到技术层面，我们分析出六个技术性层面问题：

**一：多模态输入的统一解析。** 系统的输入覆盖了关系型数据（Excel表格）、矢量图形（DWG）、文档格式（PDF/Word）和栅格图像（PNG/JPG）。这些格式在数据表征上完全不同——Excel是行列网格，DWG是几何实体集合，PDF页面可能是文本流也可能是渲染后的像素。设计一个统一的解析框架，使得每种格式都能输出一致的结构化结果，而各自的解析逻辑又能独立演进，是架构设计上的首要挑战。

**二：CAD矢量PDF的识别。** 这是本项目中技术难度最高的子问题。AutoCAD等电气设计软件导出的PDF图纸有其特殊性：文字并非以字符编码形式存储（如普通的Word转PDF），而是被渲染为贝塞尔曲线路径——从PDF文件的角度看，页面就是一堆线条和曲线的集合，没有“文字”这个概念。传统的OCR方案（如Tesseract）在这种场景下表现极差，因为复杂的电气符号、连接线、标注线会严重干扰文字区域的检测。更棘手的是，图纸中的文字方向多变——有的水平、有的垂直、有的旋转一定角度与设备符号对齐——进一步增加了识别难度。我们评估后认为，传统OCR路线对于CAD矢量PDF难以达到可用精度，需要引入具有视觉理解能力的大模型来解决。

**三：物料名称的语义归一化。** 同一元器件在不同来源中的名称差异不仅是拼写层面的，更涉及语义层面。例如“MCCB-3P-250A-36kA”、“塑壳断路器 NSX250F”、“250A 断路器 3极”和“Molded Case Circuit Breaker 250A”指向同一类物料，但从字符串角度看相似度并不高。简单的编辑距离算法无法可靠匹配这类变体。

**四：跨源数据的一致性校验。** 当同一柜号在DWG图纸、Excel清单和Word说明中都有出现时，需要判断这些来源的信息是否一致。例如，图纸标注柜号为“AA1-01”，而清单写的是“AA1-1”——这是同一个柜子吗？图纸标注柜型为“进线柜”，清单中对应的额定电流是2000A——这个配置是否合理？这类跨源校验既需要模糊匹配能力（处理柜号命名的微小差异），也需要业务规则知识（判断柜型与电流的匹配关系）。

**五：置信度的量化与表达。** 自动识别结果的置信度不能是一个笼统的数字，而应该是多维度的合成。例如，一条从Excel清单中抽取的物料记录的置信度可能来自：表头匹配的确定性、单元格内容的完整性、归一化匹配的得分。不同的置信度来源需要可分解、可追溯，才能让用户理解“为什么系统认为这条记录可信度低”，从而有针对性地复核。

**六：资源限制问题。** 如果方案纯粹依赖大规模模型推理，不仅部署成本高昂，也难以在比赛现场复现。因此，我们的策略是“规则优先，模型兜底”——确定性场景用高效的传统算法，只有传统算法无法处理的模糊场景才调用大模型API，且始终保持传统OCR作为最终兜底路径。

## 2.4 解决思路

基于以上分析，我们确立了“\*分层解耦 + 混合智能\*”的总体策略：

- **规则优先：**对于模式明确、边界清晰的任务（文件类型识别、Excel表头检测、字段名映射、柜型→物料模板匹配），优先使用确定性规则引擎实现。规则引擎的优势在于100%可解释、零计算成本、结果稳定可复现。
- **模型增强：**对于模式模糊、依赖语义理解的任务（物料别名匹配、CAD图纸文字识别、表格结构理解），引入大模型/视觉模型作为增强层。模型只在规则无法覆盖的情况下介入，且每次调用都要求输出结构化的、可追溯的结果。
- **人机闭环：**将自动识别的结果按置信度分层——高置信自动采纳，低置信标记待确认。人工修正操作通过审计日志完整记录，修正数据可以回灌到规则库和词典中，形成持续改进的正反馈循环。

### 三、系统架构设计

#### 3.1 总体架构

系统采用**前后端分离 + 异步任务调度 + 模块化解析层**的混合分层架构。这一架构选择的核心考量是：解析任务（尤其是DWG转换和PDF OCR）属于计算密集型且耗时的操作，不能阻塞Web请求的响应；同时，不同的解析器需要能够独立开发、独立部署、独立替换，因此必须通过接口协议解耦。

**架构说明：**最上层是Flask驱动的轻量Web界面，提供文件上传、解析进度查看、结果浏览与修正、导出的完整操作闭环。中间业务逻辑层负责请求路由和任务编排——当用户提交一批文件后，系统先通过文件分类器识别每个文件的类型，然后创建Celery异步任务分发给对应的解析器。解析层是系统的核心，每种文件格式对应一个独立的解析器实现，它们都遵守统一的**SourceParser**接口协议。这种接口隔离的设计意味着新增一种文件格式支持只需编写一个新的解析器类，无需修改任何已有代码。归一与生成层对解析结果进行后处理：物料名称规范化、BOM组装、报价计算。校验与导出层负责质量检查和结果输出。

#### 3.2 核心模块详述

| 层      | 模块      | 核心职责                                                         | 技术选型                       | 设计考量                                                           |
|--------|---------|--------------------------------------------------------------|----------------------------|----------------------------------------------------------------|
| 解析层    | Excel解析 | 自动检测表头行位置、识别列名映射、处理合并单元格、提取行级物料记录                            | openpyxl                   | Excel是最主要的输入格式，因此该模块投入最大；支持多Sheet自动发现、噪声行过滤、原始行号保留用于溯源         |
|        | DWG解析   | 通过LibreDWG将DWG转为DXF，利用ezdxf提取模型空间的TEXT/MTEXT实体，解码Unicode转义字符 | LibreDWG + ezdxf           | DWG是AutoCAD私有二进制格式，解析链路为DWG→DXF→文本提取→图层过滤→文本聚类；支持AC1015/1018格式 |
|        | PDF基础解析 | 检测页面是否包含可抽取的文本层；对于有文本层的PDF，提取表格数据                            | pdfminer + pdfplumber      | 预处理步骤：先判断PDF类型，再路由到对应处理路径                                      |
|        | PDF矢量图  | 将CAD矢量PDF页面渲染为高分辨率图片，送入视觉大模型进行理解                             | pdf2image + Vision LLM API | 最重要但尚未完全实现的模块；页面以300DPI渲染保证文字清晰度，通过精心设计的Prompt引导大模型输出结构化JSON   |
|        | Word解析  | 抽取段落文本和表格数据；从技术说明中通过正则匹配提取约束字段                               | python-docx                | 约束字段包括：品牌指定、防护等级、接地方式、使用环境等，这些约束将用于后续的校验规则                     |
| 归一与生成层 | 物料归一    | 别名精确匹配→模糊匹配→规格消歧三层回退                                         | JSON外置词典 + RapidFuzz       | 词典外置设计支持运维人员在不修改代码的情况下新增别名映射                                   |
|        | BOM生成   | 按柜号分组组装逐柜BOM；跨柜合并生成项目汇总BOM；规则补全辅材                            | 规则引擎 + 聚合策略                | 去重确保同一物料不重复计数                                                  |

| 层   | 模块   | 核心职责                       | 技术选型     | 设计考量                           |
|-----|------|----------------------------|----------|--------------------------------|
|     | 报价计算 | 按优先级查找价格→计算行小计→逐柜汇总→项目总价   | 价格优先级策略  | 价格后置：不在解析阶段做价格推断，避免价格逻辑与识别逻辑耦合 |
| 校验层 | 质量校验 | 执行7类规则检查，输出带定位信息的校验报告      | 规则链      | pending_*标记体系确保不确定项不被静默丢弃      |
| 导出层 | 结果导出 | 输出结构化JSON和7个Sheet的Excel工作簿 | openpyxl | JSON用于程序化集成，Excel用于人工审阅和汇报     |
| 协同层 | Web壳 | 提供上传、浏览、编辑、导出、审计追溯的完整Web界面 | Flask    | 单文件可启动，无前端构建依赖，降低部署门槛          |

3.3 技术选型原则

我们的技术选型遵循四个原则，这些原则直接影响了架构中每个模块的实现方式：

- 一、MVP优先：在项目启动阶段，我们不试图一次性支持所有文件格式和所有业务规则。而是先聚焦于最高频、最核心的路径——Excel清单解析→BOM生成→导出——用最短时间跑通可运行闭环，验证业务逻辑的正确性。之后再逐步扩展DWG、PDF、Word等格式的支持。这个策略确保我们在每个时间节点都有一个可演示、可评估的产物，而不是在漫长的开发周期结束后才第一次看到结果。
- 二、接口隔离：每种解析器都实现统一的SourceParser协议（定义在interfaces.py中），包含supports(input\_path)和parse(input\_path)两个方法。这种设计的好处是：新增一种格式支持时，只需添加一个新的解析器类并注册到解析器注册表中，已有代码完全不需要修改——这是"对扩展开放，对修改封闭"原则的直接体现。
- 三、依赖可选：OCR引擎、PDF处理库、大模型SDK等重型依赖通过Python的lazy import机制加载。如果运行环境中缺少某个依赖，系统不会崩溃，而是优雅回退到占位实现并给出明确的提示信息。这样设计的原因是：这些系统级依赖的安装在不同操作系统上差异很大，我们不想因为一个可选依赖的缺失导致整个系统无法启动。

四、数据模型设计

4.1 设计理念

数据模型的设计遵循一个核心理念：**每条数据都携带其来源证据**。在传统的自动化系统中，输出通常只包含结果本身——例如"AA1柜 断路器 3个"。然而在工业报价场景中，结果的正确性需要被验证，而验证的唯一途径是追溯到原始资料。因此，我们的数据模型从底层就内建了"来源追溯"和"置信度"两个维度。

4.2 核心实体说明

**ProjectDocument** 是整个系统的数据入口。当一个项目文件夹被提交后，系统首先扫描所有文件，对每个文件进行类型识别和初步解析，生成一个ProjectDocument实例。它的files字段列出所有发现的文件路径，metadata字典记录了解析过程的元信息——例如对于PDF文件，会记录"是否检测到文本层"、"提取到几个表格"、"OCR是否被应用"等。这个索引是后续所有处理步骤的起点。

**CabinetRecord** 代表项目中识别到的一台电气柜。它的核心字段包括：cabinet\_no（柜号）、cabinet\_type（柜型）、rated\_current（额定电流）、dimensions（外形尺寸）、circuit\_count（回路数）。每条柜体记录都附带sources列表——它是一个来源引用的数组，记录了这个柜体的信息分别来自哪些文件、哪些页面、哪些行。confidence字段则是一个0到1之间的浮点数，表示系统对这条柜体记录的总体置信度。

**MaterialRecord / BomLine（物料/BOM行）** 是系统最核心的数据实体。每条BOM行描述一个物料在某个柜体中的使用情况：物料的原始名称（`material_name`）、归一化后的规范名称（`normalized_name`）、规格型号（`spec`）、单位（`unit`）、数量（`quantity`）、品牌（`brand`）、来源依据（`source`）、识别置信度（`confidence`），以及风险标签列表（`risk_tags`）。特别重要的是`source`字段——它使得用户可以在Web界面上点击一条物料行，直接看到"这条数据来自项目B的Excel清单第42行"或"这条数据是从系统图第3页的OCR结果中识别的"。

**QuoteLine（报价行）** 在项目汇总BOM的基础上增加了价格维度。每条报价行包含：`unit_price`（单价）、`subtotal`（行小计 = 数量 × 单价）、`price_source`（价格来源）、`price_confidence`（价格置信度）。价格来源的优先级策略贯彻了"人工确认优先"的原则——当用户手动修改了一个价格后，这个修改会被记录在审计日志中，且后续的重新计算不会覆盖人工确认的价格。

**ValidationIssue（校验问题）** 记录系统自动发现的质量问题。每条问题包含：`issue_type`（问题类型）、`severity`（严重程度）、`location`（精确定位）、`details`（问题描述和上下文信息）、`suggestion`（修正建议）。这些问题的定位信息足够精确，使得用户可以从校验报告直接跳转到对应的BOM行或源文件位置进行处理。

**AuditLog（审计日志）** 记录每一次人工修正操作。包含：`record_id`（被修改的记录标识）、`action`（操作类型）、`before`（修改前的值）、`after`（修改后的值）、`user_rationale`（用户填写的修正理由）、`timestamp`（操作时间戳）。审计日志的设计服务于两个目的：一是提供修改的可追溯性，二是为未来的规则学习提供训练样本。

|                          |                       |
|--------------------------|-----------------------|
| ProjectDocument          | – 项目资料解析索引            |
| — project_name           | – 项目名称                |
| — files[]                | – 文件清单                |
| — metadata               | – 解析状态/文本层检测/表格计数     |
| CabinetRecord            | – 柜体记录                |
| — cabinet_no             | – 柜号                  |
| — cabinet_type           | – 柜型（进线柜/出线柜/MCC柜...） |
| — rated_current          | – 额定电流                |
| — dimensions             | – 外形尺寸                |
| — circuit_count          | – 回路数                 |
| — sources[]              | – 来源依据（文件/页/行）        |
| — confidence             | – 识别置信度               |
| MaterialRecord / BomLine | – 物料/BOM行             |
| — material_name          | – 物料名称（原始）            |
| — normalized_name        | – 归一化名称               |
| — spec                   | – 规格型号                |
| — unit                   | – 单位                  |
| — quantity               | – 数量                  |
| — brand                  | – 品牌                  |
| — source                 | – 来源依据                |
| — confidence             | – 置信度                 |
| — risk_tags[]            | – 风险标签（长交期/缺价等）       |
| QuoteLine                | – 报价行                 |
| — unit_price             | – 单价                  |
| — subtotal               | – 行小计                 |
| — price_source           | – 价格来源（人工/价格表/样例/缺价）  |



|                     |                |
|---------------------|----------------|
| └─ price_confidence | ─ 价格置信度        |
| ValidationIssue     | ─ 校验问题         |
| └─ issue_type       | ─ 问题类型         |
| └─ severity         | ─ 严重程度         |
| └─ location         | ─ 定位（柜号/行号/文件） |
| └─ suggestion       | ─ 修正建议         |
| AuditLog            | ─ 审计日志         |
| └─ record_id        | ─ 目标记录         |
| └─ action           | ─ 操作（修改/确认/回滚） |
| └─ before/after     | ─ 变更前后值        |
| └─ user_rationale   | ─ 修正理由         |

## 五、核心算法方案

### 5.1 多格式解析流水线

解析流水线是整个系统的入口。当用户提交一个包含多种文件的目录后，系统首先通过文件分类器识别每个文件的类型——这一步不仅检查文件扩展名（`.xlsx`、`.dwg`等），还通过magic number验证文件的实际内容类型，防止扩展名与内容不一致的情况。识别完成后，每个文件被路由到对应的格式解析器，解析器输出标准化的`ProjectDocument`对象。下面详述每种格式的解析策略。

**Excel解析：**Excel是主元器件清单的主要载体，也是当前系统投入最大的解析模块。与其他简单的“读列名取值”方案不同，我们的Excel解析器需要处理真实业务场景中的多种复杂情况：首先，表头行的位置不固定——有的清单表头在第一行，有的前面有几行标题和空行，有的表头跨两行。系统通过扫描前N行，检测每行中业务关键词的命中密度来自动定位表头行。其次，列名不统一——同样的“元器件名称”字段，在不同清单中可能叫“设备名称”、“元件名称”、“名称及规格”、“Description”等。系统通过预定义的列名聚类规则将不同命名的列映射到统一的目标字段。再次，合并单元格在工程清单中极为常见——例如一个柜号可能跨多行合并，系统通过openpyxl的合并单元格API获取合并区域，然后将合并值向下填充，确保每行都有完整的柜号信息。最后，噪声行通过规则过滤，保留的行附带原始的Excel行号，用于后续的溯源和校验定位。

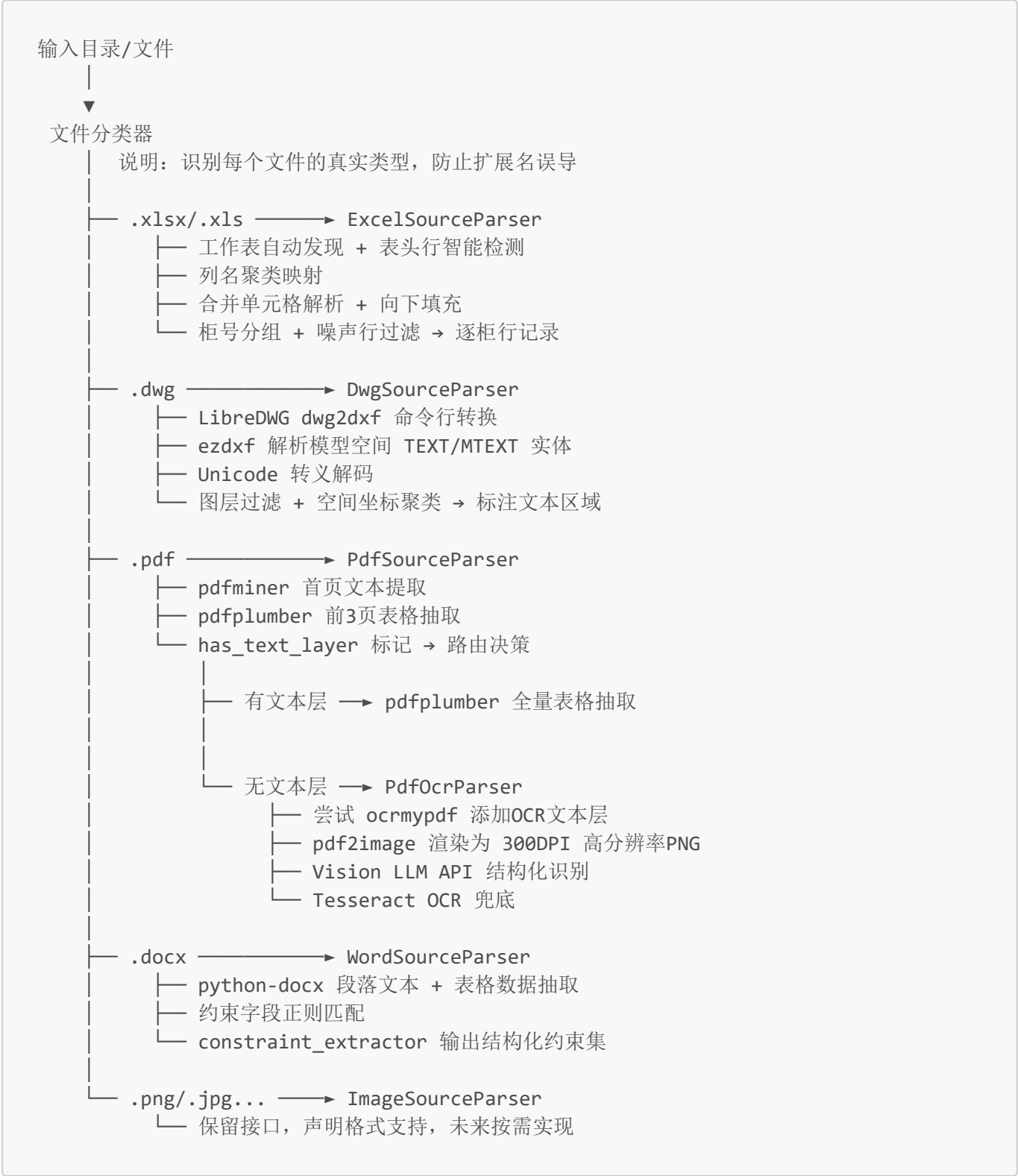
**DWG解析：**AutoCAD的DWG格式是私有二进制格式，没有公开的解析规范。我们的策略是利用开源工具LibreDWG中的`dwg2dxf`命令将DWG转换为相对开放的DXF文本格式，然后使用Python的`ezdxf`库读取DXF文件中的模型空间实体。在DXF模型空间中，文字标注以TEXT或MTEXT实体的形式存在，每个实体包含文本内容、插入点坐标、所在图层等属性。系统遍历所有TEXT/MTEXT实体，提取文本内容，并对MTEXT中的Unicode转义序列进行解码。提取后的文本按图层分组、按空间坐标聚类——距离相近的文本被认为是同一个标注区域的内容。这种方法的局限性在于只能提取显式的文字标注，无法理解电气符号的语义，但对于获取柜号、柜型、额定电流等关键文本信息是足够的。当前方案仅支持AC1015和AC1018格式（AutoCAD 2000-2007）

**PDF解析：**PDF文件的处理是所有格式中最复杂的，因为PDF的来源和内部结构差异巨大。系统的处理策略是“先检测、再路由”：首先使用`pdfminer`提取PDF首页的文本（仅一页，速度快），判断是否存在可读的文本层。如果检测到丰富的文本层，说明这是一个原生文档PDF，直接使用`pdfplumber`进行表格抽取——`pdfplumber`能较好地还原PDF中的表格行列结构。如果文本层为空或极少，说明这是一个扫描件或CAD矢量PDF——两者在外观上都是“图片”，但本质不同：扫描件是像素图像，CAD矢量是线条路径。对于扫描件，可以尝试`ocrmypdf`添加OCR文本层后再抽取，或直接使用Tesseract对渲染后的页面图片进行OCR。

**Word解析：**Word文档（`.docx`）作为Office Open XML格式，结构相对规整。系统使用`python-docx`库解析段落和表格。对于技术配置说明书这类文档，除了通用的段落抽取外，还通过`constraint_extractor`模块进行

约束字段的定向识别：使用正则表达式匹配品牌约束、防护等级、接地方式、使用环境等信息。抽取出的约束以键值对形式存储，供后续的校验规则使用——例如，如果约束中指定了"断路器品牌=施耐德"，而BOM中某个断路器行记录的是其他品牌，校验引擎就会生成一条品牌冲突告警。

**图片解析：**对于栅格图像格式，当前仅提供占位骨架，声明格式支持但不执行实际OCR。这是有意为之：在现有样例数据中，图片格式的输入极少，且OCR的质量高度依赖图像分辨率和拍摄条件，投入产出比不高。骨架的存在保证了接口完整性，未来可按需接入OCR或视觉大模型。



5.2 物料归一化算法



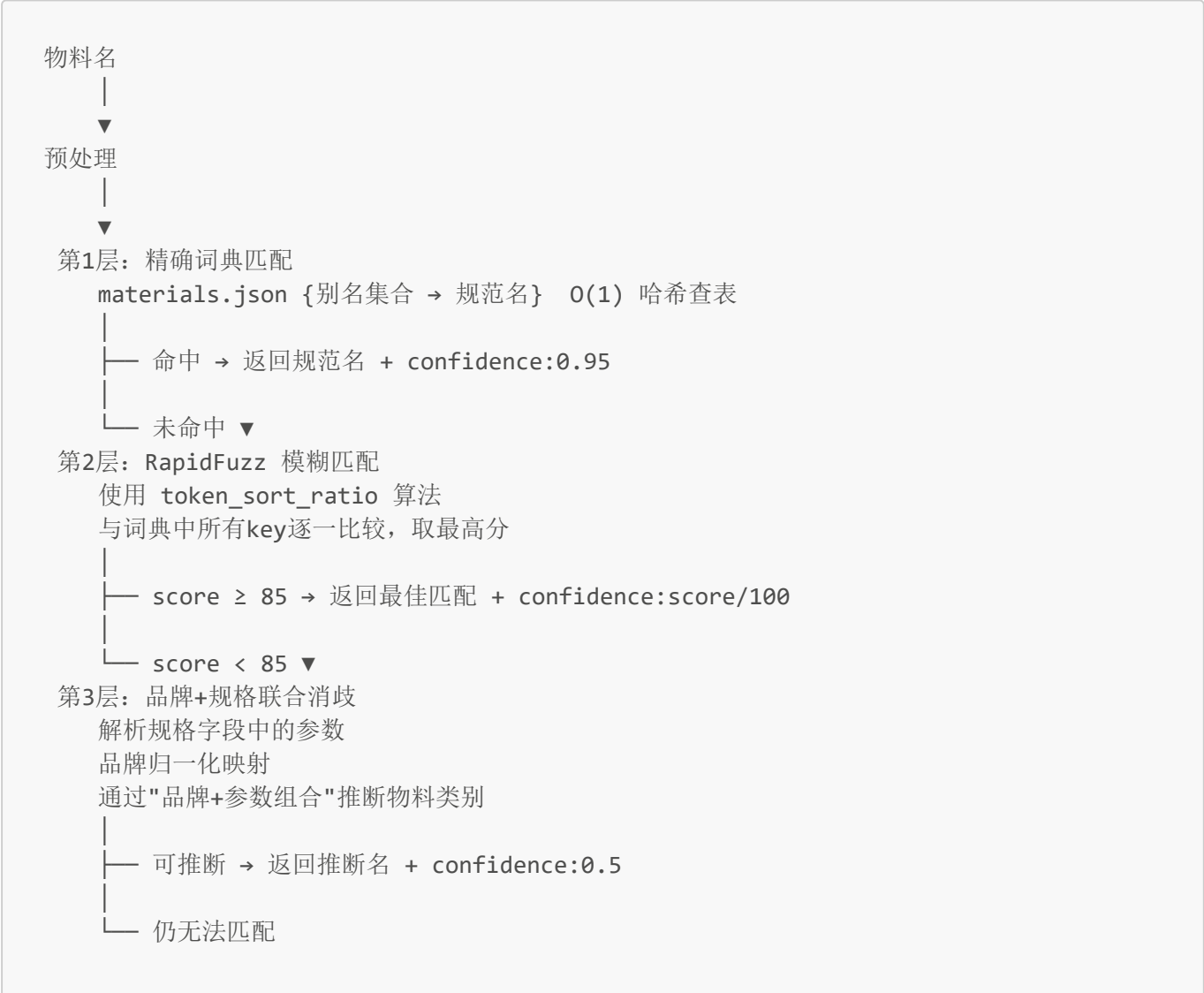
物料归一化是保证BOM质量的关键环节。不同来源对同一物料的命名方式差异巨大：中英文混用、词序颠倒、品牌型号替代物料名、缩写与全称等等。如果归一化做不好，汇总BOM中就会出现大量实质相同但被当作不同物料的冗余行，导致采购数量分散、报价不准确。

我们的归一化方案采用**分层匹配、逐级回退**的策略。这个设计背后的思想是：大多数物料可以通过简单的查表解决，少数需要模糊匹配，极少数需要品牌和规格级别的联合推理。算法的效率取决于每一层能拦截多少比例。

**第一层：精确词典匹配。**系统维护一个外置的JSON词典文件 (`materials.json`)，其中包含85条物料别名映射和38条品牌别名映射。如果命中，直接返回规范名称，置信度设为0.95。这一层的时间复杂度是O(1) (哈希查找)，几乎不消耗计算资源。

**第二层：RapidFuzz模糊匹配。**如果第一层未命中，系统使用RapidFuzz库 (基于C++的高性能模糊匹配引擎) 计算物料名与词典中所有key的相似度。我们选择RapidFuzz而非更常见的fuzzywuzzy，原因是前者速度快约10倍且在长文本上表现更好。当最佳匹配得分 $\geq 85$ 时，采纳该匹配，置信度设为 $\text{score}/100$ 。如果得分 $< 85$ ，进入第三层。

**第三层：品牌+规格联合消歧。**这一层处理最复杂的情况：物料名既不在词典中，模糊匹配也找不到足够相似的候选项。此时系统尝试从规格型号中提取结构化参数，结合品牌归一化结果，通过"品牌+规格参数组合"来推断物料类型。例如，如果一条记录的规格字段包含"250A/3P/36kA"且品牌为"施耐德"，即使物料名写得很不规范，系统也能推断这大概率是一台塑壳断路器。如果三层都无法确定，物料保留原始名称，置信度设为0.3，并附加`pending_material`标记供人工确认。



- 保留原始名称 + confidence:0.3 + pending\_material 标记
- Web界面中醒目标示，引导用户手动确认

## 5.3 BOM 生成与汇总

BOM生成分为两个阶段：逐柜BOM组装和项目汇总BOM合并。

**逐柜BOM生成：**从解析阶段获得的物料行记录，首先按cabinet\_no字段分组——属于同一台柜子的物料行被聚集到一起。在每组内部，对物料行执行三键去重：归一化名称+规格型号+品牌，三者完全相同的行合并为一行，数量累加。这一步解决的是"同一柜子内同种物料在原始清单中被拆成多行"的问题。去重后，系统调用规则引擎进行辅材补全——根据柜型、额定电流等级、接地方式等参数，从预定义的规则表中查找该柜型通常需要的辅材，并添加到BOM中。这些补全的物料行会被标记来源为"规则推算"，置信度设为0.7。最后，每条BOM行都标记其来源类型：Excel抽取、DWG识别、Word约束推断、规则补全或人工修正。

**项目汇总BOM：**将所有柜体的逐柜BOM跨柜合并，同样按"归一化名称+规格+品牌"三键去重，数量跨柜累加。与逐柜BOM不同的是，汇总BOM中每条物料额外保留了一个source\_cabinets字段（列出所有包含该物料的柜号），以使用户从汇总视图下钻到具体柜体。

逐柜 BOM 生成流程：

Excel行记录 + DWG识别记录 + Word约束



按 cabinet\_no 分组



组内处理：

- ├ 三键去重合并，数量累加
- ├ 规则引擎补全辅材
- └ 标记每条行的来源类型 + 置信度



逐柜 BOM

## 5.4 报价计算

报价计算坚持"后置"原则：不在解析阶段做任何价格推断，而是在项目汇总BOM生成后，独立运行报价计算模块。

价格查找遵循四级优先级：第一优先级是**人工确认**——如果用户在之前的修正中手动输入了某个物料的价格，这个价格将被优先使用且不会被自动计算覆盖。第二优先级是**价格表**——如果项目的metadata中提供了价格表，系统从中按物料名+规格匹配价格。第三优先级是**样例价格**——系统内置了一组常见元器件的参考价格，仅供演示和最低限度的报价生成。第四优先级是**缺价占位**——如果以上三种来源都没有价格，系统不会静默跳过，而是生成一条unit\_price=0的报价行，并标记price\_source='missing'和price\_confidence=0。这样做的考虑是保留报价结构的完整性，显式提示用户需要补充价格信息。

报价计算过程是确定性的：行小计 = 数量 × 单价；柜体汇总 = 该柜所有物料行小计之和；项目总价 = 所有柜体汇总之和。计算结果输出到QuoteLine和QuoteSummary结构中。

项目汇总 BOM



逐行价格查找（按四级优先级）：

1. 人工确认价格
2. 明确价格表
3. 样例价格
4. 缺价占位



逐行计算:  $\text{subtotal} = \text{quantity} \times \text{unit\_price}$

逐柜汇总:  $\text{cabinet\_subtotal} = \sum \text{subtotal}$

项目总价:  $\text{project\_total} = \sum \text{cabinet\_subtotal}$



输出: `QuoteLine[]`+ `QuoteSummary`（汇总）

+ `missing_price` 提示列表（需要用户补充价格的物料）

## 5.5 质量校验引擎

校验引擎是保证输出质量的最后一道防线。它运行在BOM和报价生成完成之后，以规则链的形式依次执行7类检查。每种检查都会输出带精确定位信息的`ValidationIssue`记录，支持用户在Web界面中从问题列表直接跳转到对应的数据行。

**为什么是规则引擎而不是机器学习？** 校验规则的特点是边界清晰、逻辑明确——例如“柜号为空”就是字段值为空或占位符，“数量异常”可以通过历史统计的z-score检测。这些规则用机器学习不仅是大材小用，反而可能引入难以解释的误判。我们选择显式的规则链，每条规则独立运行，规则可以增删和调整优先级而不影响其他规则。同时，我们引入了`pending_*`标记体系：当一个物料无法被归一化（`pending_material`）、规格不完整（`pending_material_spec`）或品牌待确认（`pending_material_brand`）时，系统不静默丢弃这些信息，而是通过pending标记显式记录下来——这些标记既作为校验问题的来源，也为未来的人工修正和模型训练提供了明确的关注点。

## 5.6 CAD 矢量 PDF 识别方案（创新重点）

这是本项目中技术难度最高、也最能体现创新性的模块。以下详细阐述问题的本质、我们评估过的方案、以及最终选择的路线。

**问题的本质：**当电气工程师使用AutoCAD或其兼容软件完成图纸设计后，通常会将DWG文件导出为PDF格式用于分发和打印。这个导出过程有一个关键的技术细节：CAD软件在导出PDF时，为了保持图形的精确矢量特性，会将所有内容——包括文字标注——转换为PDF的矢量路径指令。从PDF文件格式的角度看，页面上没有“字符”，只有“路径”。因此，任何依赖文本层的PDF解析工具都无法使用。

**方案评估：**我们评估了三种可能的解决方案。

**方案A：**将PDF页面渲染为高分辨率像素图像，然后用传统OCR引擎识别。在我们的测试中，Tesseract对CAD图纸渲染图的识别准确率极低，原因在于图纸中的电气符号、连接线、边框线与文字交错在一起，OCR引擎无法区分“这是一条线”和“这是一个字符的笔画”。此外，图纸文字方向多变，Tesseract对旋转文字的识别能力有限。

方案B：尝试将PDF反向转换为DXF，然后走DWG解析路线。这条路在理论上可行，但实践中pstoedit的转换质量极不稳定，经常丢失文字或产生错误的几何实体，不具备实用价值。

方案C：利用具备视觉理解能力的大模型直接"看懂"图纸页面。这些模型在训练过程中已经学习了大量包含文字、图表、工程图纸的图像，能够理解图纸中的空间布局关系。经过实际测试，大模型对电气图纸的结构化信息抽取能力远超传统OCR。

**我们的混合方案：**PDF页面首先通过pdf2image以300DPI渲染为PNG图像。对于检测到文本层的PDF页面，优先使用本地的Marker工具处理，避免消耗API额度。只有无文本层的CAD矢量PDF才调用Vision LLM API。调用时，我们设计了一套电气图纸专用的System Prompt，引导模型关注特定信息，并要求以JSON Schema约束的格式输出——输出结构直接映射到系统的CabinetRecord和MaterialRecord模型，无需后处理解析。多页PDF采用asyncio异步并发请求，配合API限流控制，确保不触发服务商的速率限制。如果API调用因任何原因失败，自动回退到Tesseract OCR作为最终兜底——虽然精度较低，但保证系统在任何情况下都能给出结果而非崩溃。



## 六、测试方案

### 6.1 测试分层策略

测试体系采用经典的金字塔分层结构：底层是大量的单元测试，中层是集成测试，顶层是端到端测试。

**单元测试层：**针对每个可独立测试的模块编写，包括各个格式解析器（Excel/DWG/PDF/Word）、物料归一化引擎、校验规则链、报价计算逻辑等。单元测试的关键原则是"隔离"——不依赖外部服务、不依赖数据库、不依赖文件系统。

**集成测试层：**集成测试使用真实的样例数据（`examples/`目录），但运行在测试隔离环境中。

**冒烟测试：**一条最精简的核心路径——Excel输入→JSON输出——用于快速验证"系统是否基本可用"。冒烟测试通常<5秒运行完成，适合在每次代码变更后立即执行。

**慢速测试：**包含DWG转换和OCR的测试。这些操作耗时较长（，且依赖系统环境（需要安装LibreDWG/Tesseract），因此通过pytest的`-m slow`标记隔离，默认不运行，仅在需要时手动触发。

| 层级    | 覆盖范围             | 框架              | 当前文件数 | 运行频率   |
|-------|------------------|-----------------|-------|--------|
| 单元测试  | 各解析器/归一器/校验器独立逻辑 | pytest          | 10    | 每次代码变更 |
| 集成测试  | 模块间协作            | pytest          | 7     | 功能合入前  |
| E2E测试 | Web壳完整交互链路       | pytest          | 1     | 发版前    |
| 冒烟测试  | 核心路径最简验          | pytest          | 1     | 每次代码变更 |
| 慢速测试  | DWG转换/OCR等耗时操作   | pytest (--slow) | 2     | 按需触发   |

6.2 评估指标体系

评估指标的设计体现了比赛评分的核心关注点：业务正确性、算法有效性、和系统性能。我们为每类指标设定了量化目标和计算方法，确保评估结果可复现、可对比。

| 指标类别   | 具体指标                    | 计算方法                         | 目标值         | 评分维度   |
|--------|-------------------------|------------------------------|-------------|--------|
| 字段抽取   | Precision / Recall / F1 | 以人工标注的"金标准"BOM为基线，逐字段比对      | ≥85%        | 业务正确性  |
| 柜体识别   | 柜号召回率、柜型分类准确率           | 系统输出柜体集合 vs 参考输出柜体集合         | ≥90%        | 业务正确性  |
| 物料归一   | Top-1准确率                | 抽样100条物料，人工判断归一结果是否正确        | ≥85%        | 算法能力   |
| 端到端BOM | 项目级 Recall / Precision  | 系统BOM行 vs 参考BOM行，按名称+规格+柜号匹配 | ≥80%        | 业务正确性  |
| 报价准确性  | 项目总价相对偏差率               |                              | 项目总价 - 参考总价 | / 参考总价 |
| 校验检测   | 已知问题检出率                 | 在样例数据中人工注入已知错误，统计检出比例        | ≥90%        | 算法能力   |
| 处理性能   | 端到端处理时延                 | 从文件上传到全部结果导出的墙钟时间            | ≤5分钟        | 性能效率   |

七、可视化与交互方案

7.1 Web 演示壳设计

我们考虑一个基于Flask的轻量级Web界面，覆盖从文件上传到结果导出的完整操作闭环。

操作流程设计遵循"渐进式展示"原则——用户在每个步骤看到的都是当前阶段最相关的信息，而非一次性堆砌所有数据：

上传资料 → 运行解析 → 浏览项目概览 → 深入柜体/BOM → 查看校验报告 → 人工修正  
→ 重新导出

## 7.2 人机协同设计

人机协同是本次比赛明确要求的核心能力之一。我们的设计考虑是"**AI做初筛，人工做确认**"——系统完成所有能自动化的部分，但把最终的决策权留给用户，同时确保用户的每一次操作都被完整记录和可逆。

**置信度分层与可视化：**自动识别的结果按照置信度分为三个等级。高置信（ $\geq 0.8$ ，绿色标记）的记录通常是规则引擎精确匹配或词典命中，准确度很高，用户可以批量确认。中置信（ $0.5-0.8$ ，黄色标记）的记录是模糊匹配或规则推断的结果，建议用户浏览确认。低置信（ $< 0.5$ ，红色标记）的记录标志着系统无法确定——比如物料归一化失败、OCR识别文字模糊、规格字段不完整——这些是用户需要优先处理的目标。

**修正操作闭环：**用户在Web界面中对任何字段的修改，都会触发一个完整的后处理流程。具体来说：用户修改了某条BOM行的品牌字段后，系统首先将修改记录写入AuditLog，然后重新运行校验引擎，最后刷新页面展示更新后的校验结果。如果用户修改了物料名称，还会重新触发归一化引擎，因为新的名称可能匹配到不同的规范名。

**审计与回滚：**每次修正都可通过审计日志查询。用户可以在任何时候查看一条记录的历史变更——谁在什么时候改了什么、为什么改。如果发现修正是错误的，可以回滚到修改前的值。

### 自动识别结果

- 高置信（绿， $\geq 0.8$ ）→ 自动采纳，用户可展开查看来源详情
- 中置信（黄， $0.5-0.8$ ）→ 建议人工浏览，界面上以黄色高亮引导
- 低置信（红， $< 0.5$ ）→ 标记为待确认，界面上以红色高亮优先展示

### 人工修正链路：

- 点击行内编辑按钮 → 修改字段值 → 填写修正理由
  - AuditLog.write
  - 触发重归一化
  - 触发重校验
  - 触发重导出
  - 修正记录可随时查询、可回滚

### 审计日志查询：

```
AuditLog.filter(record_id=xxx)
→ [{action: "修改品牌", before: "正泰", after: "施耐德",
    rationale: "技术规范要求使用施耐德", timestamp: "2026-06-14 15:30"}]
```

## 7.3 结果导出设计

**JSON格式：**输出完整的结构化数据，包括项目元信息、所有柜体记录、逐柜BOM明细、项目汇总BOM、报价清单、校验问题列表和审计日志。JSON格式保留了所有来源引用和置信度字段，适合下游系统的程序化集成。



**Excel格式：**为人工审阅和汇报场景设计。每个Sheet承担不同的查看目的——项目经理关注Cabinets和Issues，采购关注Summary和Quote，工程师关注BOM明细和来源溯源。列宽已预设为可读宽度，置信度低的单元格以条件格式着色，便于快速扫读。

| Sheet名称      | 内容      | 目标读者 |
|--------------|---------|------|
| Project      | 项目元信息   | 所有人  |
| Cabinets     | 柜体清单    | 项目经理 |
| BOM          | 逐柜BOM明细 | 工程师  |
| Summary      | 项目汇总BOM | 采购人员 |
| Quote        | 逐行报价    | 商务人员 |
| QuoteSummary | 报价汇总    | 项目经理 |
| Issues       | 校验问题清单  | 质量审核 |

## 八、创新点阐述

**一：多模态混合解析统一流水线。** 不同于大多数方案只聚焦于单一输入格式，本系统构建了统一的解析框架，通过SourceParser接口协议将Excel、DWG、PDF、Word、图片五种异构格式纳入同一个流水线。每种格式的解析器独立实现、独立演进，但输出统一的数据结构。这种设计使得新增格式支持时不需要修改任何已有代码，具备良好的可扩展性。在工业软件领域，这种"非标准资料结构化"能力是系统能否落地的关键——实际业务中资料的类型远比赛样例更丰富，接口的可扩展性直接决定了系统的生命力。

**二：大模型驱动的CAD矢量PDF识别。** 这是本方案在技术路线上最显著的差异化优势。传统OCR方案在处理CAD矢量PDF时几乎不可用——原因在于CAD软件导出的PDF将文字渲染为矢量路径，与图纸中的电气符号和连接线混合在一起，OCR引擎无法区分。我们引入视觉大模型来理解图纸页面的语义。通过精心设计的System Prompt和JSON Schema约束输出，大模型的识别结果可以直接映射到系统的结构化数据模型，实现端到端的自动化。同时，我们设计了混合路由策略和多重兜底机制，保证了系统的鲁棒性。

**三：分层归一化与pending标记体系。** 物料归一化不是简单的字符串匹配，而是"精确词典→模糊匹配→参数消歧"的三层递进策略。每一层处理不同难度的变体，确保效率与覆盖面的平衡。更重要的是，我们引入了pending\_\*标记体系——对于归一化失败、规格不确定、品牌待确认的物料，系统不静默丢弃或强行匹配，而是显式标记为待确认状态。这些标记在Web界面中以醒目的方式展示，引导用户关注，同时为未来的规则学习和模型训练提供了明确的样本来源。这种"知道什么不知道"的设计哲学体现了工业软件中对确定性和可解释性的追求。

**四：全链路可追溯的证据链。** 系统中的每一条数据——从柜体记录到BOM行到报价行到校验问题——都附带完整的来源追溯信息。这种可追溯性不仅服务于用户的信任建立，更服务于工业场景中常见的审计和合规需求。

**五：闭环的人机协同与持续学习机制。** 本系统设计了完整的"修正→重处理→审计→学习"闭环。每次人工修正都会触发关联模块的重新计算，确保修正后的结果经过同等严谨的处理。所有修正操作记录在审计日志中，支持查询、回滚和模式分析。

### 开源方案参考

以下开源项目为本系统的PDF识别方案提供了重要的技术参考和备选路径：



| 项目              | 核心能力                                    | 参考链接                                                                                                   | 在本系统中的角色                             |
|-----------------|-----------------------------------------|--------------------------------------------------------------------------------------------------------|--------------------------------------|
| Marker          | PDF→Markdown本地SOTA<br>(0.18s/页，表格89.4%) | <a href="https://github.com/datalab-to/marker">github.com/datalab-to/marker</a>                        | 有文本层PDF的本地<br>处理方案                   |
| olmOCR          | AI2出品的7B VLM文档OCR<br>(Apache 2.0)       | <a href="https://github.com/allenai/olmOCR">github.com/allenai/olmOCR</a>                              | 大模型OCR方案参<br>考，benchmark<br>baseline |
| Vision<br>Parse | 商业LLM API的PDF解析封装<br>库                  | <a href="https://github.com/iamarunbrahma/vision-parse">github.com/iamarunbrahma/vision-<br/>parse</a> | API调用封装模式参<br>考                      |
| OCRFlux         | 3B参数VLM，支持跨页表格<br>合并                    | <a href="https://github.com/chatdoc-com/OCRFlux">github.com/chatdoc-com/OCRFlux</a>                    | 本地VLM方案备选                            |
| Surya<br>OCR    | 90+ 语言OCR + 布局检测<br>(20k star)          | <a href="https://github.com/datalab-to/surya">github.com/datalab-to/surya</a>                          | 传统OCR增强方案参<br>考                      |